

Agent Traces with MLFlow and Red Hat OpenShift AI

Version 1, 2026-06-30

Home

Goal

This course teaches how to deploy MLFlow, as included with Red Hat OpenShift AI 3.4; How to collect traces of an agentic application, using the MLFlow SDK with Python; and how to visualize those traces using the Red Hat OpenShift AI web interface.

This course is intended to require learners between 4 hours to one working day to complete.



Work In Progress

Objectives

On completing this course, you should be able to:

- Understand how MLFlow relates to agentic applications and to Red Hat OpenShift AI.
- Deploy a development environment with MLFlow
- Collect traces from an agentic application using MLFlow

Audience

- Software developers working on agentic applications and agentic harnesses.
- Red Hat and Partner roles that perform demonstrations, PoC and PoV of MLFlow with Red Hat OpenShift AI, such as Solutions Architects.
- Red Hat and Partner roles that implement and support MLFlow with Red Hat OpenShift AI, such as Services Consultants, Technical Account Managers, and Customer Support Engineers.

Prerequisites

This course assumes that you have the following experience:

- Familiarity with generative AI and agentic application concepts.
- Familiarity with the basic operation of Red Hat OpenShift AI.
- Python programming skills are recommended but not required, because this training provides sample code which is ready to run.

Classroom environment

The demo catalog entry [Red Hat OpenShift AI 3.4](#) provides an SNO OpenShift cluster, backed by an AWS instance with a single GPU, with a predeployed llama-32-3b-instruct model which is capable of making tool calls.

This course uses that demo entry as-is, and any additional components which are required by

either MLFlow itself or any of the course activities will be configured by learners, as part of course activities, by using sample Python code and Kubernetes manifests provided in a public Git repository.

Other sources of information

For documentation about MLFlow with Red Hat OpenShift AI, see the [Red Hat OpenShift AI product documentation](#).

For upstream MLFlow documentation, see the [MLFlow project documentation](#).

Author

Fernando Lozano

Training Content Architect

Red Hat - Product Portfolio Marketing & Learning

Lab Environment

Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. Log in to the [RHDP portal](#)
2. Search for the catalog entry: **Red Hat Openshift AI 3**
3. Click the catalog entry in the search results.
4. On the catalog page, click **Order**.
5. Fill out the required details in the order form and accept the defaults for other fields. You can use the smaller AWS instance type.
6. Review the warning at the bottom of the form and check:
I confirm that I understand the above warnings.
7. Await the email notification stating that your demo environment is ready and providing its access credentials and URLs.

Important Notes:

- This lab may take approximately **1:30** hours to become ready.
- You can also retrieve access information directly from the RHDP portal.
- This lab uses GPU-enabled instances on AWS, which are in short supply. It requires multiple attempts to provision or restart.



Do **not** follow the instructions/showroom from the demo catalog entry. This course reuses that demo but does not require that you perform any of its activities.

You will use the preprovisioned instance of Red Hat OpenShift AI as-is, and deploy MLFlow and sample agentic applications as part of this course activities.

How to Access Your Lab via the RHDP Portal:

1. On the RHDP portal, click **Services** option in the left-hand menu.
2. Select your lab from the services listings on the right-hand side of the page to view access details.

RHDP Portal Links

- RedHat associates: <https://demo.redhat.com/>
- RedHat partners: <https://partner.demo.redhat.com/>

Understanding MLFlow and Red Hat OpenShift AI

Goal

Understand how MLFlow relates to agentic applications and to Red Hat OpenShift AI.

This chapter introduces MLFlow in the context of Red Hat OpenShift AI, focusing on features relevant to generative AI and agentic applications. You'll learn about MLFlow's architecture, supported features, and how it integrates with RHOAI components.

Introduction to MLFlow in Red Hat OpenShift AI

Estimated reading time: 5 minutes.

Objective

Understand MLFlow's features for agentic applications and how it integrates with Red Hat OpenShift AI.



Not started yet.

MLFlow Features in RHOAI

Lorem ipsum

Focus on Generative AI Applications

Lorem ipsum

MLFlow Experiments

Lorem ipsum

What's Next

Now that you understand MLFlow's role in RHOAI and agentic applications, the next section provides a quiz to verify your understanding of key MLFlow concepts.

Quiz: MLFlow Concepts Review

Estimated reading time: 5 minutes.

Objective

Verify your understanding of MLFlow concepts and terminology.

Interactive Assessment

What's next

Now that you've verified your understanding of MLFlow concepts, the next chapter explores how to deploy MLFlow in your RHOAI environment for development purposes.

Lab: Explore the RHOAI Instance

Objective

Explore the predeployed Red Hat OpenShift AI instance to familiarize yourself with its configuration and with the new Gen AI Studio feature from 3.x releases.



Work In Progress

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment. Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard

Instructions

In this lab, you will log in to the RHOAI dashboard and explore its main features and components to understand the platform's capabilities.

1. Access the RHOAI Dashboard.
 - a. Navigate to the RHOAI dashboard URL of your demo environment.
 - b. Log in using the credentials provided for your demo environment.
2. Explore the precreated project.

Your RHOAI instance contains a model which is already deployed in a regular project named `my-first-model` — which is **not** an AI project.

That model takes up all GPU capacity available to your RHOAI instance, so any change to the model deployment requires that you stop it and restart after making changes.

- a. On the left pane, click **AI hub** > **Models** and select the **Deployments** tab.
- b. On the **Project** combo box, type `my` and select `my-first-model`.



Depending on the current page of the RHOAI dashboard, it may be necessary to first select **All projects** before you can search and select the `my-first-model` project.

- c. Verify that there is a model deployment named `llama-32-3b-instruct` and its status is **Ready**.
 - d. Click **View**, under the Endpoints column for the model row, and verify that it provides only an internal endpoint. This means the model cannot be accessed from outside of its OpenShift cluster, unless you change its model deployment.
3. Try the model with an Gen AI Studio Playground.
- a. On the left pane, click **Gen AI studio › Playground**.
 - b. On the **Project** combo box, type `my` and select `my-first-model`.



The project previously selected in one page of the RHOAI dashboard may not be the current selection in another page of the RHOAI dashboard.

- c. If you do not see a **Configure** pane, click menu:Settings. On the **Configure** pane, select the **Model** tab and verify that the selected model is named `llama-32-3b-instruct`. Ignore any prefix to the model name.
 - d. On the right pane, click the **Send a message...** text and type a prompt, for example `What is RHEL?`
 - e. You may get different answers, if you attempt the same prompt again, but they would all be correct and reasonable, hopefully.
4. Verify that there are no RHOAI dashboard pages related to MLFlow.
- a. On the left pane, expand the **Gen AI studio** category, which contains entries named **Playground** and **AI asset endpoints**.
- After you enable MLFlow, there will be additional entries here.
- b. Click **Gen AI studio › AI asset endpoints** and select the **MCP servers** tab.
- Verify that there are no MCP servers configured for use by Gen AI studio playgrounds.
- c. On the left pane, expand the **Develop & train** category, which contains entries named **Fature store**, **Pipelines**, **Jobs**, and **Evaluations**.
 - d. On the top right, click the **Applications** icon, which is close to the **Info (?)** icon, and verify that there is **not** an entry named **MLFlow**.

After completing these steps, you have verified that you have a functional RHOAI instance with a pre-deployed model, which is available only for internal access, and that you can submit prompts to the model using an RHOAI playground.

In addition, you have also verified that MLFlow is not installed on your RHOAI instance.

What's next

In the next chapter, you will learn how to deploy MLFlow on Red Hat OpenShift AI to enable experiment tracking and model management capabilities.

Deploying MLFlow for Development

Goal

Deploy a development environment with MLFlow

This chapter guides you through deploying MLFlow in Red Hat OpenShift AI for development purposes. You'll learn about MLFlow's architecture, how it integrates with RHOAI, and gain hands-on experience deploying and verifying your MLFlow instance.

MLFlow Architecture and RHOAI Integration

Estimated reading time: 5 minutes.

Objective

Understand MLFlow's architecture and how it integrates with Red Hat OpenShift AI.



Not started yet.

MLFlow Components

Lorem ipsum

RHOAI Integration

Lorem ipsum

Development vs Production Deployments

Lorem ipsum

What's Next

Now that you understand MLFlow's architecture and integration with RHOAI, the next section provides hands-on experience deploying MLFlow in a development environment.

Lab: Deploy MLFlow on Red Hat OpenShift AI

Estimated reading time: 5 minutes.

Objective

Deploy and verify a development instance of MLFlow in Red Hat OpenShift AI.



Work In Progress

Before you begin

You need access to the predeployed Red Hat OpenShift AI (RHOAI) instance provided in your lab environment. Ensure you have:

- Access credentials for the RHOAI dashboard
- A web browser to access the RHOAI dashboard
- Access credentials for the OpenShift web console

Instructions

You will enable MLFlow on your Data Science Cluster singleton resource and deploy a development instance of the MLFlow server. Then you will create a simple Jupyter notebook to test connectivity to the MLFlow server by means of the MLFlow SDK for Python.

1. Enable MLFlow on RHOAI.
 - a. Open the OpenShift web console and log in as a user with `cluster-admin` rights.
 - b. In the left navigation pane, select **Ecosystem > Installed Operators**. Then click **Red Hat OpenShift AI**.
 - c. In the Red Hat OpenShift AI operator page, click the **Data Science Cluster** tab. Then click the **default-dsc** resource.
 - d. In the `default-dsc` resource page, select the **YAML** tab. In the text editor, search for the `spec.components` field, and set its `mlflowoperator.managementState` field to `true`.

```
...
spec:
  components:
    mlflowoperator:
      managementState: Managed
    kserve:
      managementState: Managed
      rawDeploymentServiceConfig: Headless
...
```

- e. Click **Save** and wait until you see a confirmation message similar to:

```
alert:default-dsc has been updated to version 1186240
```

- f. If you get an error message, fix the errors on your YAML and try saving again.
- g. Alternatively, you could run the following `oc patch` command, which is provided by the [RHOAI product docs](#):

```
$ oc patch datasciencecluster default-dsc --type=merge \
-p '{"spec":{"components":{"mlflowoperator":{"managementState":"Managed"}}}}'
```

2. Configure your MLflow instance.

- a. In the left navigation pane, select **Home** > **API Explorer**. Then type **MLflow** in the search field. You will see three resource types, click on **MLflow**.
- b. In the MLflow Resource details page, select the **Instances** tab and click **Create MLflow**.
- c. In the YAML text editor, delete all of its sample contents and type the following, which corresponds to the example from [Minimal development or test deployment](#) in product docs.

```
apiVersion: mlflow.opendatahub.io/v1
kind: MLflow
metadata:
  name: mlflow
  namespace: redhat-ods-applications
spec:
  storage:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 10Gi
  backendStoreUri: "sqlite:///mlflow/mlflow.db"
  artifactsDestination: "file:///mlflow/artifacts"
  serveArtifacts: true
```

- d. Click **Create** and, in the MLflow details page, wait until the **Available** condition is **True** and the **Progressing** condition is **False**.

3. Verify MLFlow is available from RHOAI

- a. Open your RHOAI dash board. If it is already open, refresh your browser window.
- b. In the left navigation pane, expand **Develop & train**. Notice the new item named **Experiments (mlflow)** and click it.
- c. In the Project combo box, type **my** and select the **my-first** model project.
- d. Click **Create Experiment** and type **dummy experiment** as its name.
- e. Verify that you see the Experiments page for **dummy experiment** but there is no data on any of its pages, such as **Traces**, **Sessions**, or **Prompts**.
- f. Creating an empty experiment is sufficient to prove that your MLflow instance can connect to its database and artifacts storage.
- g. You can, alternatively, open the upstream MLFlow web interface by clicking the applications menu, in the top navigation bar, close to the help icon, and selecting **MLflow**. You should not need it: all functionality you need for this course is available from the RHOAI dashboard.

4. Test connectivity to MLFlow from a workbench.

- a. In the left navigation pane, click **Projects** and select **All projects** in the projects combo box.

Then type `my` in the filter by name field and click `my-first-project` in the list of projects.

- b. In the Projects page for `my-first-model`, select the **Workbenches** tab and click menu:Create workbench.
- c. In the Create workbench page, type `test-mlflow` for the **Name** field and, in the Workbench image field, select the `Jupyter | Minimal | CPU` image.
- d. Leave all other fields with their default values and click **Create workbench**.
- e. In the Workbenches list, wait until the row for the `test-mlflow` workbench displays a `Ready` state and then click `test-mlflow` to open its Jupyter Lab in a new browser tab.
- f. Click the **Python 3.12** Notebook icon to create an empty notebook.
- g. In the first cell, type:

```
! pip install mlflow[kubernetes]
```

- h. In the top toolbar from the notebook, click the play icon, with the tooltip **Run this cell and advance**.
- i. Wait until the MLFlow SDK and all its dependencies are loaded from the RHOAI index into the workbench.
- j. After the `pip` command finishes, type the following on the second cell of your notebook:

```
import os
import mlflow

os.environ["MLFLOW_TRACKING_AUTH"]="kubernetes-namespaced"
mlflow.set_tracking_uri("CHANGE_ME")

print(mlflow.list_workspaces())
```

- k. Switch to your RHOAI dashboard page, which should still be in the Experiments page. Notice, close to the top left, the **Launch MLflow** link. Right-click it and copy its URL to the clipboard.
- l. Switch to your jupyter lab browser tab and replace, in the second notebook cell, the `CHANGE_ME` text with the URL on your clipboard.
- m. In the top toolbar from the notebook, click the play icon. The results should an array containing a single `Workspace` object named `my-first-model`.

This notebook takes advantage of the integration between the RHOAI dashboard and its managed instance of the MLFlow server. It connects using the service account of the `test-mlflow` workbench, which is authorized to access the MLFlow workspace corresponding to its `my-first-model` project.

- n. If you wish, you can remove the unnamed notebook, or if you prefer you can save it, rename and reuse for the next activity.

After completing these steps, you have verified that you have a minimal but functional MLFlow instance, to which you can connect from applications running inside your RHOAI cluster and store data from your experiments.

What's next

Now that you have a working MLFlow deployment, the next chapter explores how to collect traces from agentic applications using the MLFlow SDK.

Collecting Traces from Agentic Applications

Goal

Collect traces from an agentic application using MLFlow

This chapter teaches you how to use the MLFlow SDK to collect traces from agentic applications. You'll learn why tracing is essential for troubleshooting agents, how to organize traces in experiments and runs, and gain hands-on experience adding MLFlow tracing to your code.

Tracing Agentic Applications

Estimated reading time: 5 minutes.

Objective

Understand why tracing is essential for agentic applications and how MLFlow organizes trace data.



Not started yet.

Why Developers Need Traces

Lorem ipsum

MLFlow SDK and Trace Calls

Lorem ipsum

Experiments and Runs

Lorem ipsum

Significant Events in Traces

Lorem ipsum

What's Next

Now that you understand tracing concepts, the next section provides hands-on experience modifying an agent to record traces using the MLFlow SDK.

Lab: Modify an Agent to Record Traces

Estimated reading time: 5 minutes.

Objective

Add MLFlow tracing to an agentic application and visualize the generated traces.



Not started yet.

Before you begin

Describe requirements, such as customer portal accounts, Red Hat Developer free sub, already configured machines, access to container registries, etc.

Instructions

High-level summary of what you will perform in this lab, but more detailed than the objective.

1. Prototype an agent using the GenAI playground.
 - a. Something to do, using a graphical or web UI. Click **Save As** to save your work.
 - b. More things to do
2. Generate code and add MLFlow tracing calls.
 - a. Something to do, using a shell.

```
$ command --options fixed-arguments variable-arguments
output of the command
highlight something in the output
```

- b. More things to do. Press `Ctrl` + `C` to interrupt.

Summary of what you accomplished.

What's next

Now that you've added basic tracing to an agent, the next section explores how to trace MCP tool calls to capture more detailed information about agent behavior.

Lab: Trace MCP Tool Calls from an Agent

Estimated reading time: 5 minutes.

Objective

Deploy an MCP server and capture its tool invocations in MLFlow traces.



Not started yet.

Before you begin

Describe requirements, such as customer portal accounts, Red Hat Developer free sub, already configured machines, access to container registries, etc.

Instructions

High-level summary of what you will perform in this lab, but more detailed than the objective.

1. Deploy an MCP server.
 - a. Something to do, using a graphical or web UI. Click **Save As** to save your work.
 - b. More things to do
2. Add the MCP server to your agentic application.
 - a. Something to do, using a shell.

```
$ command --options fixed-arguments variable-arguments  
output of the command  
highlight something in the output
```

- b. More things to do. Press `Ctrl` + `C` to interrupt.

Summary of what you accomplished.

What's next

Congratulations! You've completed this course and learned how to deploy MLFlow, collect traces from agentic applications, and visualize trace data. You can now apply these skills to troubleshoot and improve your own agentic applications.